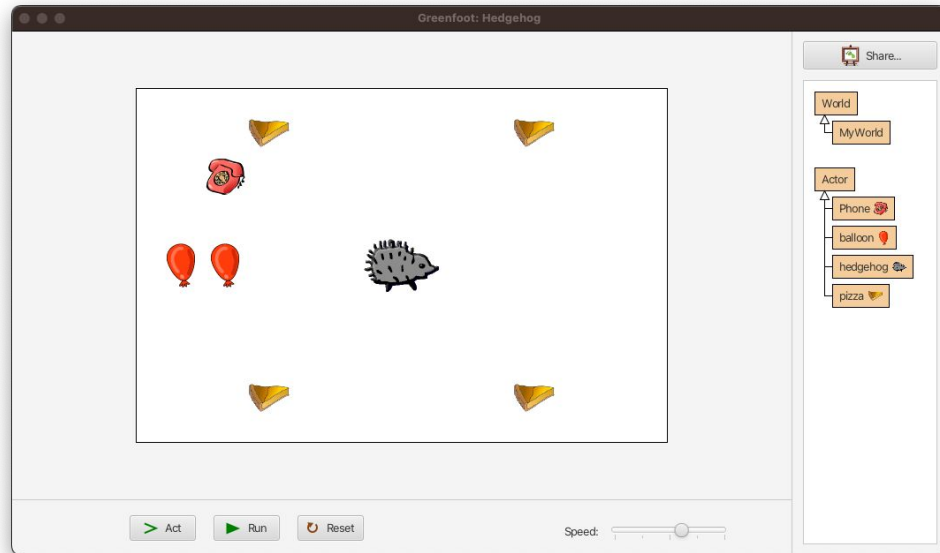


# Greenfoot and Classes

# Let's break down some features of Hedgehog

## Hedgehog was our first Greenfoot program



# What kind of variables are speed, Lives, Points?

```
public class hedgehog extends Actor
{
    private int speed = 2;
    private int Lives = 3;
    private int Points = 0;
```

- A. instance variables
- B. static variables
- C. formal parameters
- D. local variables
- E. actual parameters

# What kind of variables are speed, Lives, Points?

```
public class hedgehog extends Actor  
{
```

```
    private int speed = 2;  
    private int Lives = 3;  
    private int Points = 0;
```

**A. instance variables**

B. static variables

C. formal parameters

D. local variables

E. actual parameters

- Inside the class
- Outside any methods
- Do not have the word static

# What kind of vars are p, hh, ba, ba2, etc?

```
public MyWorld() {  
    super(600, 400, 1);  
    addObject(p, 100, 100);  
    addObject(hh, 300, 200);  
    addObject(ba, 100, 200);  
    addObject(ba2, 50, 200);  
    addObject(pi1, 150, 50);  
    addObject(pi2, 450, 50);  
    addObject(pi3, 450, 350);  
    addObject(pi4, 150, 350);  
}
```

- A. instance variables
- B. static variables
- C. formal parameters
- D. local variables
- E. actual parameters

# What kind of vars are p, hh, ba, ba2, etc?

```
public MyWorld() {  
    super(600, 400, 1);  
    addObject(p, 100, 100);  
    addObject(hh, 300, 200);  
    addObject(ba, 100, 200);  
    addObject(ba2, 50, 200);  
    addObject(pi1, 150, 50);  
    addObject(pi2, 450, 50);  
    addObject(pi3, 450, 350);  
    addObject(pi4, 150, 350);  
}
```

- A. instance variables
- B. static variables
- C. formal parameters
- D. local variables
- E. actual parameters**

# Now let's take a look at the ShootingGame



```
public class Background {  
    public Background() {  
        super(700, 388, 1);  
        Cannon cannon = new Cannon();  
        addObject(cannon, 63, 327);  
        Bird bird = new Bird();  
        addObject(bird, 528, 137);  
        Bird bird2 = new Bird();  
        addObject(bird2, 579, 282);  
        Bird bird3 = new Bird();  
        addObject(bird3, 412, 119);  
        Bird bird4 = new Bird();  
        addObject(bird4, 332, 197);  
        Bird bird5 = new Bird();  
        addObject(bird5, 453, 218);  
        Bird bird6 = new Bird();  
        addObject(bird6, 298, 295);  
        Bird bird7 = new Bird();  
        addObject(bird7, 443, 306);  
        Bird bird8 = new Bird();  
        addObject(bird8, 566, 212);  
        Bird bird9 = new Bird();  
        addObject(bird9, 511, 339);  
        Bird bird10 = new Bird();
```

# What kind of method is Background?

- A. instance method
- B. static method
- C. constructor
- D. scientific method
- E. method to my madness



```
public class Background {  
    public Background() {  
        super(700, 388, 1);  
        Cannon cannon = new Cannon();  
        addObject(cannon, 63, 327);  
        Bird bird = new Bird();  
        addObject(bird, 528, 137);  
        Bird bird2 = new Bird();  
        addObject(bird2, 579, 282);  
        Bird bird3 = new Bird();  
        addObject(bird3, 412, 119);  
        Bird bird4 = new Bird();  
        addObject(bird4, 332, 197);  
        Bird bird5 = new Bird();  
        addObject(bird5, 453, 218);  
        Bird bird6 = new Bird();  
        addObject(bird6, 298, 295);  
        Bird bird7 = new Bird();  
        addObject(bird7, 443, 306);  
        Bird bird8 = new Bird();  
        addObject(bird8, 566, 212);  
        Bird bird9 = new Bird();  
        addObject(bird9, 511, 339);  
        Bird bird10 = new Bird();
```

# What kind of method is Background?

- A. instance method
- B. static method
- C. constructor**
- ~~D. scientific method~~
- ~~E. method to my madness~~

- No return type
- Matches name of class

```
public Background() {  
    super(700, 388, 1);  
  
    Cannon cannon = new Cannon();  
    Bird bird = new Bird();  
    Bird bird2 = new Bird();  
    Bird bird3 = new Bird();  
  
    // rest of code
```

What kind of vars are cannon, bird1, bird2, bird3 when created?

- A. instance variables
- B. static variables
- C. formal parameters
- D. local variables
- E. actual parameters

```
public Background() {  
    super(700, 388, 1);  
  
    Cannon cannon = new Cannon();  
    Bird bird = new Bird();  
    Bird bird2 = new Bird();  
    Bird bird3 = new Bird();  
  
    // rest of code
```

What kind of vars are cannon, bird1, bird2, bird3 when created?

- A. instance variables
- B. static variables
- C. formal parameters
- D. local variables**
- E. actual parameters

local variables created in the  
Background() constructor

```
public Background() {  
    super(700, 388, 1);  
  
    Cannon cannon = new Cannon();  
    Bird bird = new Bird();  
    Bird bird2 = new Bird();  
    Bird bird3 = new Bird();  
  
    addObject(cannon, 63, 327);  
    addObject(bird, 528, 137);  
    addObject(bird2, 579, 282);  
    addObject(bird3, 412, 119);  
}
```

Wait, but local vars go away  
once the method is done!

How does our program know  
about them once the constructor  
method ends?

```
public Background() {  
    super(700, 388, 1);  
  
    Cannon cannon = new Cannon();  
    Bird bird = new Bird();  
    Bird bird2 = new Bird();  
    Bird bird3 = new Bird();  
  
    addObject(cannon, 63, 327);  
    addObject(bird, 528, 137);  
    addObject(bird2, 579, 282);  
    addObject(bird3, 412, 119);  
}
```

Wait, but local vars go away once the method is done!

Cannon and birds are all passed to `addObject()` which keeps track of them.

The `Background` class (a type of `World`) *encapsulates* everything added with `addObject`.

We can look at the source code of Greenfoot!

<https://github.com/denisecase/Greenfoot-source-361/blob/main/greenfoot/src/greenfoot/World.java#L421>

addObject() puts the Actor parameter into a bunch of its own instance variables. So the World knows about every Actor that is added.

```
public void splatter()  
{  
    if (isTouching(Bird.class)){  
        Krieger krieger = new Krieger();  
        getWorld().addObject(krieger, getX(), getY());  
        removeTouching(Bird.class);  
    }  
    if (isAtEdge()) {  
        getWorld().removeObject(this);  
    }  
}
```

Consider  
`isTouching()` and  
`removeTouching()`

These methods work because  
`addObject()` stores all of the  
instances into various instance  
variables that `isTouching()` and  
`removeTouching()` can access

```
public void act(){  
    if (Greenfoot.isKeyDown("left")){  
        setRotation (getRotation() - 5);  
    }  
    if (Greenfoot.isKeyDown("right")){  
        setRotation (getRotation() + 5);  
    }  
    if  
    ("space".equals(Greenfoot.getKey())){  
        fire();  
    }  
}
```

What type of variable  
do we think getRotation  
and setRotation are  
changing?

- A. instance variable
- B. static variable
- C. formal parameter
- D. local variable
- E. actual parameter



```
public void act(){  
    if (Greenfoot.isKeyDown("left")){  
        setRotation (getRotation() - 5);  
    }  
    if (Greenfoot.isKeyDown("right")){  
        setRotation (getRotation() + 5);  
    }  
    if  
    ("space".equals(Greenfoot.getKey())){  
        fire();  
    }  
}
```

What type of variable  
do we think getRotation  
and setRotation are  
changing?

- A. instance variable**
- B. static variable
- C. formal parameter
- D. local variable
- E. actual parameter

Actor probably looks something like this:

```
public class Actor {  
    private int rotation = 90;  
    public int getRotation() {  
        return rotation;  
    }  
    public void setRotation(int degrees) {  
        while (degrees < 0) degrees += 360;  
        rotation = degrees % 360;  
    }  
}
```

The other methods in Actor are similar

[move\(\) changes x, y coordinates](#)

setImage() changes the image

and so on

# Greenfoot source code for Actor

<https://github.com/denisecase/Greenfoot-source-361/blob/91907870d62af85d49128206f3fc1f9f18f52810/greenfoot/src/greenfoot/Actor.java#L191>

There's a lot in here, but let's just focus on `getRotation()` and `setRotation()`, `move()`, and `setImage()`

These change instance variables

```
public void act(){  
    if (Greenfoot.isKeyDown("left")){  
        setRotation (getRotation() - 5);  
    }  
    if (Greenfoot.isKeyDown("right")){  
        setRotation (getRotation() + 5);  
    }  
    if  
    ("space".equals(Greenfoot.getKey()))  
    {  
        fire();  
    }  
}
```

What type of methods  
are isKeyDown() and  
getKey() ?

- A. instance method
- B. static method
- C. constructor
- D. method of modern love
- E. none of the above

```

public void act(){
    if (Greenfoot.isKeyDown("left")){
        setRotation (getRotation() - 5);
    }
    if (Greenfoot.isKeyDown("right")){
        setRotation (getRotation() + 5);
    }
    if
    ("space".equals(Greenfoot.getKey()))
    {
        fire();
    }
}

```

We are invoking these methods directly on the class Greenfoot. That means they are static. We don't need an instance first.

What type of methods are `isKeyDown()` and `getKey()`?

- A. instance method
- B. static method**
- C. constructor
- D. method of modern love
- E. none of the above

What type of variables are image, rank, suit?

```
public class Card extends Actor{  
    GreenfootImage image;  
    int rank;  
    String suit;
```

- A. instance variable
- B. static variable
- C. formal parameter
- D. local variable
- E. actual parameter

What type of variables are image, rank, suit?

```
public class Card extends Actor{  
    GreenfootImage image;  
    int rank;  
    String suit;
```

- A. instance variable**
- B. static variable
- C. formal parameter
- D. local variable
- E. actual parameter



# What type of variables are rank and suit?

```
public Card(int rank, String suit)
{
    this.rank = rank;
    this.suit = suit;
    String filename = toRankString(rank) + suit + ".png";
    this.image = new GreenfootImage("images/cards/" + filename);
    setImage(image);
}
```

- A. instance variable
- B. static variable
- C. formal parameter
- D. local variable
- E. actual parameter

# What type of variables are rank and suit?

```
public Card(int rank, String suit)
{
    this.rank = rank;
    this.suit = suit;
    String filename = toRankString(rank) + suit + ".png";
    this.image = new GreenfootImage("images/cards/" + filename);
    setImage(image);
}
```

- A. instance variable
- B. static variable
- C. formal parameter**
- D. local variable
- E. actual parameter

That's all I got done for today

Suppose we are making a new platformer

Where should we store the health of the player?

A. instance var inside

<https://pixelfrog-assets.itch.io/pixel-adventure-1>